

# GPT-4o 기반 멀티모달 AI 시스템 구축

## 이미지 분석에서 음성 합성까지의 심층 기술

### 1. 서론: 멀티모달 인공지능의 패러다임 전환과 GPT-4o의 위상

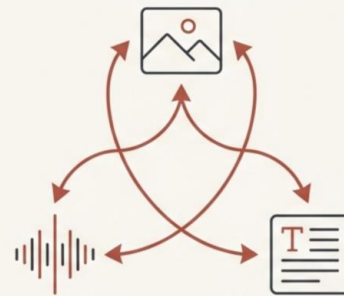
#### 패러다임의 전환: 텍스트를 넘어 세상을 인지하는 AI

##### 과거 (LLM - Large Language Model)



GPT-3와 같이 텍스트 데이터만 학습하여 언어 패턴을 이해하던 시대.

##### 현재 (LMM - Large Multimodal Model)



GPT-4o('o'는 omni를 의미)와 같이 시각, 청각, 언어를 통합적으로 인지하고 처리하는 시대. 단순한 이미지-텍스트 변환을 넘어 이미지 내의 복합적인 맥락, 사물 간의 관계, 잠재된 분위기까지 인간 수준으로 이해.

#### 1.1 대형 언어 모델(LLM)을 넘어선 대형 멀티모달 모델(LMM)의 시대

인공지능 기술의 발전사는 데이터 처리 양식(Modality)의 확장과 궤를 같이합니다. 텍스트 데이터만을 학습하여 언어적 패턴을 생성하던 GPT-3와 같은 대형 언어 모델(Large Language Model, LLM)의 시대를 지나, 이제는 시각(Vision), 청각(Audio), 언어(Text)를 통합적으로 인지하고 처리하는 대형 멀티모달 모델(Large Multimodal Model, LMM)의 시대가 도래했습니다.<sup>1</sup> 이러한 변화의 중심에 있는 GPT-4o('o'는 omni를 의미)는 단순히 이미지를 텍스트로 변환하는 수준을 넘어, 이미지 내의 복합적인 맥락, 사물 간의 관계, 잠재된 감정선, 그리고 추상적인 분위기까지 인간 수준으로 이해하는 능력을 보여줍니다.

본 강의 문서는 『GPT-4o를 이용한 AI 이미지 분석가』 커리큘럼을 기반으로, GPT-4o의 비전(Vision) 기능과 음성 합성(Text-to-Speech, TTS) 기술을 결합하여 실무적인 AI 에이전트를 구축하는 전 과정을 심도 있게 다룹니다. 특히 단순한 API 호출 실습을 넘어, 이미지 데이터의

바이너리 처리 원리, 토큰 경제학(Tokenomics)에 기반한 비용 최적화 전략, 그리고 현존하는 비전 모델의 기술적 한계와 극복 방안을 포괄적으로 분석합니다. 이는 수강생들이 단순한 코더(Coder)를 넘어 AI 아키텍트(Architect)로서의 역량을 갖추도록 하는 데 목적이 있습니다.

## 1.2 강의의 기술적 목표와 범위

본 강의는 크게 세 가지의 기술적 마일스톤을 달성하는 것을 목표로 합니다.

첫째, 비정형 데이터인 이미지를 머신러닝 파이프라인에 주입하기 위한 데이터 전처리 기술(Base64 Encoding)과 네트워크 전송 프로토콜을 이해합니다.

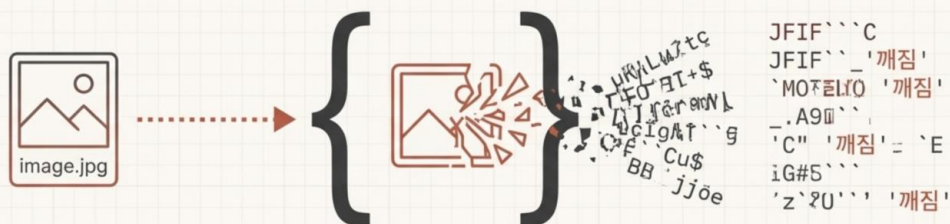
둘째, GPT-4o 비전 API의 매개변수 튜닝을 통해 이미지 분석의 정밀도와 비용 효율성을 제어하는 고급 프롬프트 엔지니어링 기술을 습득합니다.

셋째, 시각 정보를 텍스트로 변환하고, 이를 다시 청각 정보로 변환하는 멀티모달 체인(Multimodal Chain)을 구축하여, 자동화된 교육 콘텐츠 생성 시스템(Image-to-Quiz-to-Speech)을 구현합니다.

## 2. 이미지 데이터 처리의 컴퓨터 과학적 원리와 전송 프로토콜

### 2.1 바이너리 데이터와 텍스트 프로토콜의 불일치 문제

#### 첫 번째 관문: 어떻게 이미지를 AI에게 '말'해줄 것인가?



##### 문제 상황 (Problem):

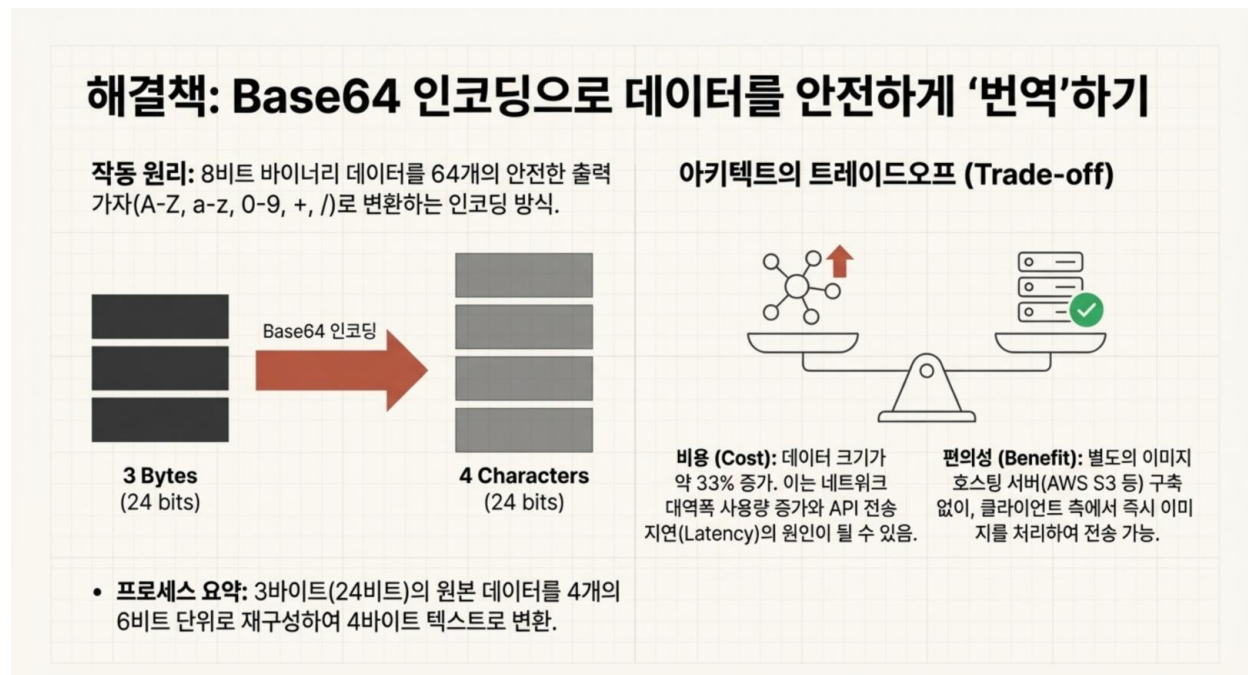
- 이미지 파일: **바이너리** (0과 1의 집합).
- API 통신 (JSON): **텍스트 기반** 포맷.
- 충돌: 두 형식은 근본적으로 **호환되지 않아**, 바이너리 데이터를 텍스트 채널로 직접 전송 시 **데이터가 손상되거나 '깨짐'** 현상 발생.

##### 핵심 질문: 이 구조적 불일치를 어떻게 해결할 것인가?

GPT-4o 와 같은 최신 AI 모델과 통신하기 위해서는 주로 HTTP 기반의 RESTful API 를 사용하며, 데이터 교환 포맷으로는 JSON(JavaScript Object Notation)을 채택합니다. JSON 은 근본적으로 텍스트 기반의 포맷이기에, 0 과 1 로 이루어진 바이너리(Binary) 형태의 이미지 파일(JPG, PNG 등)을 직접 담을 수 없는 구조적 한계가 존재합니다.

이미지 파일을 텍스트 에디터로 강제로 열었을 때 알아볼 수 없는 깨진 문자들이 나타나는 것은, 이미지의 바이너리 패턴이 텍스트 인코딩(ASCII 나 UTF-8) 체계와 호환되지 않기 때문입니다. 이러한 바이너리 데이터를 텍스트 기반의 통신 채널(이메일의 SMTP, 웹의 HTTP JSON 등)로 안전하게 전송하기 위해 고안된 것이 바로 Base64 인코딩입니다.

## 2.2 Base64 인코딩의 매커니즘과 오버헤드 분석



Base64 는 8 비트(1 바이트) 단위의 바이너리 데이터를 6 비트 단위로 재구성하여, 64 개의 출력 가능 문자(A-Z, a-z, 0-9, +, /)로 변환하는 인코딩 방식입니다.

| 단계     | 설명           | 기술적 상세                                        |
|--------|--------------|-----------------------------------------------|
| 1. 입력  | 원본 이미지 데이터   | 8 비트 바이트 스트림 (예: 01010100 01010111...)        |
| 2. 버퍼링 | 24 비트 단위 그룹화 | 3 개의 바이트(3 x 8 bits = 24 bits)를 하나의 처리 단위로 묶음 |
| 3. 분할  | 6 비트 단위 재분할  | 24 비트 데이터를 4 개의 6 비트 청크로 나눔                   |
| 4. 매핑  | 인덱스 변환       | 각 6 비트 값(0~63)을 Base64 인덱스 테이블의 문자로 변환        |
| 5. 패딩  | 길이 보정        | 입력 데이터가 3 바이트로 나누어떨어지지 않을 경우<br>= 문자로 패딩 처리   |

이 과정에서 3 바이트의 원본 데이터가 4 바이트의 텍스트 데이터로 변환되므로, 필연적으로 데이터 크기가 약 33% 증가하는 오버헤드(Overhead)가 발생합니다. 이러한 크기 증가는 네트워크 대역폭 사용량 증가와 API 전송 지연(Latency)의 원인이 될 수 있습니다. 그러나 별도의 이미지 호스팅 서버(AWS S3 등)를 구축하여 URL 을 생성하는 복잡성을 제거하고, 클라이언트 측에서 즉시 이미지를 처리하여 보낼 수 있다는 편의성 때문에 소규모 이미지 분석이나 실시간 에이전트 구축에서는 Base64 방식이 널리 사용됩니다.

## 2.3 Python 을 이용한 이미지 전처리 파이프라인 구현

로컬 이미지를 API 가 수용 가능한 형태로 변환하기 위해 다음과 같은 전처리 함수를 구현하고 있습니다.

---

```
# image_explanation.ipynb
```

```
import base64
```

```
def encode_image(image_path):
```

```
    # 'rb' 모드는 파일을 텍스트가 아닌 바이너리로 읽겠다는 명시적 선언
```

```
    with open(image_path, "rb") as image_file:
```

```
        # 읽어들이는 바이트 객체를 Base64 로 인코딩 후, JSON 전송을 위해 UTF-8 문자열로 디코딩
```

```
        return base64.b64encode(image_file.read()).decode("utf-8")
```

---

이 함수는 운영체제 상의 이미지 파일 경로를 받아, 이를 메모리에 바이너리 형태로 로드(Load)하고, 즉시 인코딩하여 문자열 객체로 반환합니다. 이 문자열은 추후 API 요청 본문(Payload)의 image\_url 필드에 data:image/jpeg;base64,... 헤더와 함께 삽입되어, 모델이 이를 이미지 데이터로 인식하게 합니다.

## 3. GPT-4o 비전 API의 아키텍처와 프롬프트 엔지니어링

### 3.1 멀티모달 메시지 구조와 컨텍스트 주입

GPT-4o의 챗 컴플리션(Chat Completion) API는 텍스트와 이미지를 동시에 수용할 수 있는 유연한 메시지 구조를 가집니다. 기존 LLM이 content 필드에 단순 문자열만을 받았다면, GPT-4o는 content 필드에 텍스트 객체와 이미지 객체로 구성된 리스트(Array)를 받습니다.<sup>3</sup>

이러한 구조는 사용자가 "이 이미지에 대해 설명해 주세요"라는 텍스트 지시사항(Instruction)과 시각적 컨텍스트(Image Data)를 하나의 턴(Turn)에 결합하여 전달할 수 있게 합니다. 모델은 입력된 순서대로 데이터를 처리하며, 텍스트가 지시하는 바에 따라 이미지의 특정 피처(Feature)에 집중하는 어텐션(Attention) 메커니즘을 작동시킵니다.

### 3.2 인터넷 URL을 활용한 이미지 분석 실습

강의의 첫 번째 실습에서는 언스플래쉬(Unsplash)와 같은 공개 이미지 저장소의 URL을 사용하여 분석을 수행합니다. 이 방식은 로컬 전처리 과정이 필요 없으며, 트래픽 부담을 오픈 AI 서버와 이미지 호스팅 서버로 분산시킨다는 장점이 있습니다.

실습 코드에서 image\_url 파라미터에 웹 링크를 직접 삽입함으로써, GPT-4o는 해당 URL에 접속하여 이미지를 다운로드하고 내부적으로 분석 파이프라인을 가동합니다. 결과적으로 "검은색 운동복을 입은 여성이 파란색 그릇에 달걀을 휘젓고 있다"는 구체적인 행동 묘사와 "여유롭고 행복한 표정"이라는 감정적 추론 결과까지 도출해냅니다. 이는 모델이 픽셀 정보에서 객체(여성, 그릇, 달걀)를 탐지(Detection)하는 것을 넘어, 객체 간의 상호작용(Action)과 장면에 내포된 의미(Sentiment)를 통합적으로 이해하고 있음을 시사합니다.

#### 실습코드\_01 : # image\_explanation.ipynb

##### Cell in 1

```
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키를 가져오기

client = OpenAI(api_key=api_key) # 오픈 AI 클라이언트의 인스턴스 생성
```

```

messages = [
    {
        "role": "user",
        "content": [
            {"type": "text", "text": "이 이미지에 대해 설명해주세요."},
            {
                "type": "image_url",
                "image_url": {
                    "url": "https://images.unsplash.com/photo-1736264335247-8ec5664c8328?q=80&w=1887&auto=format&fit=crop&ixlib=rb-4.0.3&ixid=M3wxMjA3fDB8MHxwaG90by1wYWdlfHx8fGVufDB8fHx8fA%3D%3D",
                },
            },
        ],
    }
]

```

```

response = client.chat.completions.create(
    model="gpt-4o", # 응답 생성에 사용할 모델 지정
    messages=messages # 대화 기록을 입력으로 전달
)

```

response

---

## 출력결과

```
ChatCompletion(id='chatcmpl-CLRfrlf2mzt1N8af8DThbko7RXU69',
choices=[Choice(finish_reason='stop', index=0, logprobs=None,
message=ChatCompletionMessage(content='한 여성이 주방에서 요리를 준비하고 있는
모습입니다. 그녀는 검은색 운동복을 입고 있으며, 파란색 그릇에 달걀을 휘젓고 있습니다. 주방에는
프라이팬과 요리 도구들이 있습니다. 그녀는 미소를 지으며 요리를 즐기고 있는 것처럼 보입니다.',
refusal=None, role='assistant', audio=None, function_call=None, tool_calls=None,
annotations=[])]), created=1765423831, model='gpt-4o-2024-08-06', object='chat.completion',
service_tier='default', system_fingerprint='fp_37d212baff',
usage=CompletionUsage(completion_tokens=76, prompt_tokens=1119, total_tokens=1195,
completion_tokens_details=CompletionTokensDetails(accepted_prediction_tokens=0,
audio_tokens=0, reasoning_tokens=0, rejected_prediction_tokens=0),
prompt_tokens_details=PromptTokensDetails(audio_tokens=0, cached_tokens=0)))
```

## 3.3 로컬 이미지 분석과 인코딩 데이터의 활용

두 번째 실습에서는 사용자가 직접 촬영한 '망원동 빵집' 사진을 분석합니다. 인터넷에 업로드되지 않은 프라이빗 이미지는 앞서 설명한 Base64 인코딩 방식을 통해 전송해야 합니다. 이 과정에서 중요한 점은 API 가 인코딩된 문자열을 해석할 때 data:image/jpeg;base64,와 같은 프리픽스(Prefix)를 통해 미디어 타입(MIME Type)을 식별한다는 것입니다.

빵집 이미지 분석 결과, 모델은 진열된 빵의 종류(마카롱, 크루아상)뿐만 아니라, "뒤쪽에 직원들이 작업하는 모습"과 같은 배경 요소(Background Elements)까지 포착해냅니다. 이는 사용자가 미처 인지하지 못한 세부 정보(Fine-grained details)까지 AI 가 식별할 수 있음을 보여주며, CCTV 관제나 품질 관리(QC) 자동화와 같은 산업적 응용 가능성을 입증합니다.

### Cell in 2

```
import base64

# Function to encode the image
def encode_image(image_path):
    with open(image_path, "rb") as image_file:
        return base64.b64encode(image_file.read()).decode("utf-8")
```

```
image_path = "../data/images/mangwon_bakery.jpg"
```

```
# 이미지를 base64 로 인코딩
```

```
base64_image = encode_image(image_path)
```

```
print(base64_image)
```

---

출력결과 생략

### Cell 3

---

```
messages = [  
    {  
        "role": "user",  
        "content": [  
            {"type": "text", "text": "이 이미지에 대해 설명해주세요."},  
            {  
                "type": "image_url",  
                "image_url": {  
                    "url": f"data:image/jpeg;base64,{base64_image}",  
                },  
            },  
        ],  
    },  
]
```

```
response = client.chat.completions.create(  
    model="gpt-4o", # 응답 생성에 사용할 모델 지정  
    messages=messages # 대화 기록을 입력으로 전달  
)
```

response.choices[0].message.content

---

### 출력결과

---

'이 이미지는 베이커리 카운터를 보여줍니다. 여러 종류의 빵과 패스트리가 유리 진열대에 진열되어 있고, 각각의 제품 앞에는 가격표가 붙어 있습니다. 다양한 종류의 빵이 층별로 나뉘어 있으며, 사람들은 뒤편에서 일하고 있는 모습이 보입니다. 전반적으로 깔끔하고 따뜻한 분위기의 베이커리입니다.'

---

## 4. 비전 모델의 비용 구조와 해상도 제어 전략

### 4.1 토큰 기반 과금 체계와 이미지 타일링(Tiling)

### 비용 설계: 모든 픽셀에는 가격이 있다



**핵심 개념: 타일링 (Tiling)**  
모델은 이미지를 한 번에 처리하는 것이 아니라, 512x512 픽셀 크기의 "타일" 단위로 분할하여 비용을 산정.

**비용 공식**

**총 토큰 = 기본 비용(85) + (타일 당 비용(170) × 타일 개수)**

예시: 1024x1024 이미지는 4개의 타일(2x2)로 처리되어  $(170 \times 4) + 85 = 765$  토큰 소요.

**전략적 선택: `detail` 파라미터**

**`low`**  
빠르고 저렴 (85 토큰 고정).  
이미지 전체 구도, 색상 등 대략적 분석에 적합.

☒

**`high`**  
정밀하고 비쌈 (타일 수에 비례).  
이미지 내 작은 텍스트(OCR), 미세한 결함 분석에 필수적.

GPT-4o의 이미지 처리 비용은 텍스트와 마찬가지로 '토큰(Token)' 단위로 산정되지만, 그 계산 방식은 이미지의 해상도와 디테일 설정에 따라 달라지는 독자적인 메커니즘을 따릅니다.<sup>8</sup> 이미지는 한 번에 처리되는 것이 아니라, 512x512 픽셀 크기의 '타일(Tile)' 또는 '패치(Patch)' 단위로 분할되어 처리됩니다.

| 구성 요소      | 토큰 비용      | 비고                              |
|------------|------------|---------------------------------|
| 기본 오버헤드    | 85 토큰      | 이미지를 처리하기 위한 기본 비용              |
| 타일(Tile)   | 170 토큰 / 개 | 512x512 픽셀 단위 조각 하나당 비용         |
| Low Detail | 85 토큰 (고정) | 해상도와 관계없이 512x512로 리사이징 후 단일 처리 |

예를 들어, 1024x1024 크기의 이미지를 high 모드로 처리할 경우:

- 이미지는 768x768 크기로 리스케일링(Rescaling)됩니다.
- 이를 커버하기 위해 4개의 512x512 타일(2x2 배열)이 필요합니다.
- 총 비용 = (170 토큰 x 4개) + 85 토큰 = 765 토큰이 소요됩니다.

## 4.2 Detail 파라미터 튜닝: Low, High, Auto

API 호출 시 detail 파라미터를 통해 분석의 깊이와 비용을 제어할 수 있습니다.

- **Low Mode:** 이미지의 전체적인 구도, 색감, 주요 객체의 존재 여부만을 빠르게 파악할 때 사용합니다. 85 토큰이라는 저렴한 비용으로 처리가 가능하므로, 대량의 이미지를 분류(Classification)하거나 썸네일을 생성하는 작업에 적합합니다.
- **High Mode:** 이미지 내의 작은 텍스트(OCR), 그래프의 수치, 미세한 결함 등을 분석할 때 필수적입니다. 이미지를 여러 타일로 쪼개어 정밀하게 분석하므로 비용이 증가하지만 높은 정확도를 보장합니다.
- **Auto Mode:** 입력된 이미지의 크기를 보고 모델이 자동으로 모드를 결정합니다. 일반적으로 일정 크기 이상이면 High 모드로 작동합니다.

강의에서는 기본적으로 auto 설정을 사용하지만, 비용 최적화가 필요한 상용 서비스 개발 시에는 목적에 따라 low 와 high 를 명시적으로 구분하여 사용하는 전략이 필요합니다.

## 5. 비교 분석 및 심층 추론 능력 검증

### 5.1 복수 이미지 입력과 관계 추론

#### Step 1 | 이미지에서 맥락 읽어내기: 단순 묘사를 넘어 관계 추론까지

💡 **프롬프트:** "두 이미지의 차이점을 설명해 주세요."

☕ 테라로사 선릉점



**AI 분석 결과 (선릉점):**

- 어두운 조명, 나무 바닥, 원형 테이블  
→ "아늑함, 그룹 모임에 적합"

🏠 테라로사 홍대서교점



**AI 분석 결과 (홍대서교점):**

- 밝은 조명, 타일 바닥, 금속 인테리어  
→ "현대적, 개인 작업에 적합"

**인사이트:** 모델이 픽셀 정보를 넘어 공간이 주는 '심상(Atmosphere)'과 '용도(Utility)'를 추론하는 능력. 이는 인테리어 추천, 부동산 매물 비교 등 고부가가치 서비스의 기반이 됨.

GPT-4o의 강력한 기능 중 하나는 여러 장의 이미지를 동시에 입력받아 그 관계를 해석하는 능력입니다. 강의 자료의 '테라로사 카페 비교' 실습은 두 개의 서로 다른 공간(선릉점, 홍대서교점) 이미지를 제공하고 "차이점을 설명해 달라"는 고차원적인 명령을 수행합니다.

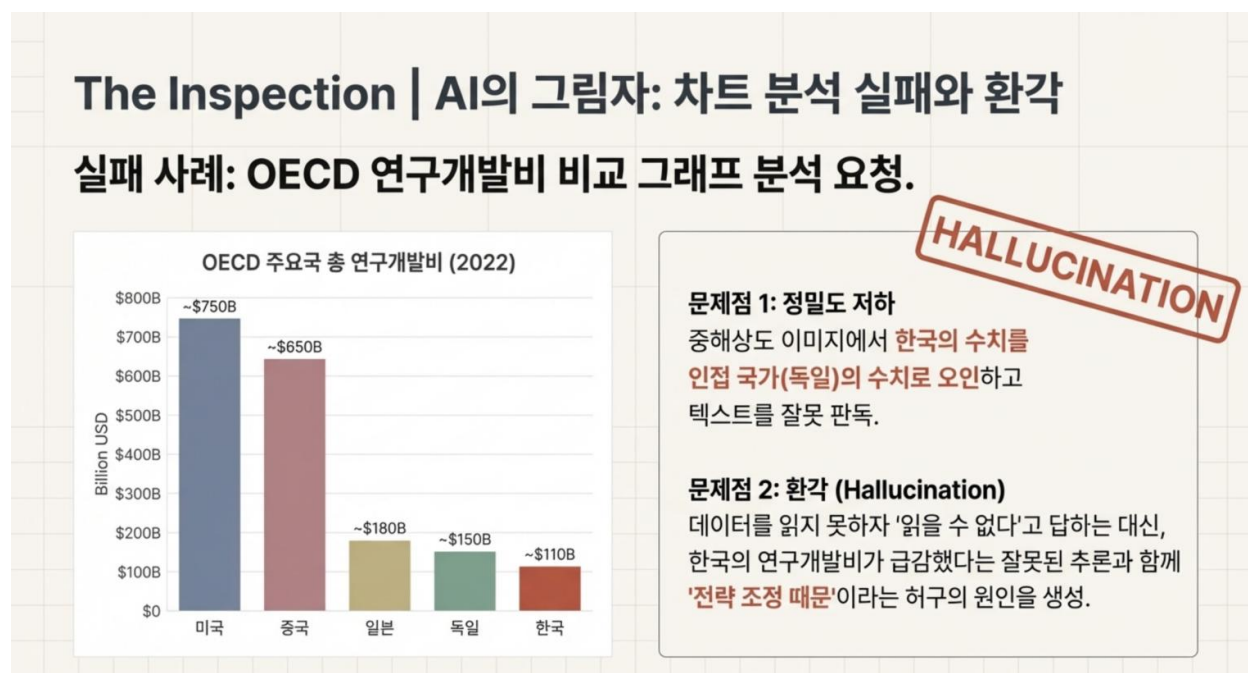
모델은 두 이미지의 시각적 특징을 각각 추출한 뒤, 이를 비교 대조(Compare and Contrast)하는 논리적 과정을 수행합니다.

- **선릉점:** 어두운 조명, 나무 바닥, 원형 테이블 → "아늑함, 그룹 모임 적합"
- **홍대서교점:** 밝은 조명, 타일 바닥, 금속 인테리어 → "현대적, 개인 작업 적합"

이러한 분석은 단순한 픽셀 정보를 넘어, 공간이 주는 '심상(Atmosphere)'과 '용도(Utility)'를 추론했다는 점에서 의미가 큼. 이는 인테리어 추천 시스템, 부동산 매물 비교 서비스, 브랜드 이미지 분석 등에 활용될 수 있는 핵심 기술입니다.

## 6. 비전 모델의 한계점 분석: 데이터 환각과 정밀도 문제

### 6.1 차트 및 그래프 분석의 실패 사례 연구



AI 모델의 능력은 완벽하지 않으며, 특히 정형 데이터가 시각화된 차트나 그래프를 해석할 때 그 한계가 명확히 드러납니다. 한국과학기술기획평가원(KISTEP)의 OECD 연구개발비 비교 그래프 분석 실패 사례를 다룹니다.

- **해상도에 따른 성능 저하:** 고해상도 이미지(large)에서는 비교적 정확한 수치를 읽어냈으나, 중해상도 이미지(medium)에서는 한국의 수치를 인접한 국가(독일)의 수치로 오인하거나, 텍스트를 잘못 판독하는 오류를 범했습니다.
- **환각(Hallucination) 현상:** 더욱 심각한 문제는 모델이 데이터를 읽지 못했을 때 "읽을 수 없다"고 응답하는 대신, 맥락상 그럴듯한 거짓 수치를 생성해낸다는 점입니다. 실습에서 모델은 한국의 연구개발비가 급감했다는 잘못된 추론을 제시하며, 그 원인으로 "전략 조정"이라는 허구의 배경 설명까지 덧붙였습니다.

### 비전 처리 한계점 확인 예제

```
oecd_rnd_2021_base64 = encode_image("../data/images/oecd_rnd_2021_large.png")
```

```
oecd_rnd_2022_base64 = encode_image("../data/images/oecd_rnd_2022_large.png")
```

```

messages = [
    {
        "role": "user",
        "content": [
            {"type": "text", "text": "첫번째는 2021 년 데이터이고, 두번째는 2022 년 데이터입니다. 이  
데이터에 대해 설명해주세요. 어떤 변화가 있었나요? 한국 중심으로 설명해주세요."},
            {
                "type": "image_url",
                "image_url": {
                    "url": f"data:image/jpeg;base64,{oecd_rnd_2021_base64}",
                },
            },
            {
                "type": "image_url",
                "image_url": {
                    "url": f"data:image/jpeg;base64,{oecd_rnd_2022_base64}",
                },
            },
        ],
    }
]

```

```

response = client.chat.completions.create(
    model="gpt-4o", # 응답 생성에 사용할 모델 지정
    messages=messages # 대화 기록을 입력으로 전달
)

```

```

response.choices[0].message.content

```

## 출력결과

---

'두 데이터는 2021 년과 2022 년의 주요 국가들의 연구개발비(R&D)와 GDP 대비 연구개발비 비중을 보여줍니다. 한국을 중심으로 주요 변화를 살펴보면 다음과 같습니다.

1. \*\*연구개발비 변화\*\*:

- 2021 년: 한국의 연구개발비는 89,282 백만 달러입니다.
- 2022 년: 한국의 연구개발비는 **91,013 백만 달러**로 증가했습니다.

2. \*\*GDP 대비 연구개발비 비중 변화\*\*:

- 연구개발비의 GDP 대비 비중은 2021 년에는 4.93%였습니다.
- 2022 년에는 5.21%로 증가하여, GDP 대비 연구개발비 투자가 상대적으로 늘어난 것을 보여줍니다.

3. \*\*전반적인 국제 비교\*\*:

- 한국의 연구개발비 규모는 세계에서 상위권 수준을 유지하며, 비중 측면에서도 다른 국가와 비교했을 때 높은 수준을 유지하고 있습니다.

이러한 변화는 한국의 연구 및 개발 분야에 대한 투자가 지속적으로 이루어지고 있으며, 경제 성장에서의 혁신의 중요성을 반영하는 것으로 볼 수 있습니다.'

---

## 해상도 낮은 모델 확인

---

```
oecd_rnd_2021_base64 = encode_image("../data/images/oecd_rnd_2021_medium.png")
```

```
oecd_rnd_2022_base64 = encode_image("../data/images/oecd_rnd_2022.png")
```

```
messages = [
```

```
{
```

```
    "role": "user",
```

```
    "content": [
```

```
        {"type": "text", "text": "첫번째는 2021 년 데이터이고, 두번째는 2022 년 데이터입니다. 이
```

```
데이터에 대해 설명해주세요. 어떤 변화가 있었나요? 한국 중심으로 설명해주세요."},
```

```
{
```

```
    "type": "image_url",
```

```
    "image_url": {
```

```
        "url": f"data:image/jpeg;base64,{oecd_rnd_2021_base64}",
```

```
    },
```

```

    },
    {
      "type": "image_url",
      "image_url": {
        "url": f"data:image/jpeg;base64,{oecd_rnd_2022_base64}",
      },
    },
  ],
}
]

```

```

response = client.chat.completions.create(
    model="gpt-4o", # 응답 생성에 사용할 모델 지정
    messages=messages # 대화 기록을 입력으로 전달
)

```

```
response.choices[0].message.content
```

---

## 출력결과

---

'이 데이터는 주요 국가의 연구개발비와 GDP 대비 연구개발비 비중을 보여주고 있습니다.

2021 년과 2022 년의 한국 데이터를 비교해보면 다음과 같은 변화가 있습니다:

### 1. \*\*연구개발비\*\*

- \*\*2021 년\*\*: 106,121 백만 US 달러
- \*\*2022 년\*\*: 87,225 백만 US 달러
- 연구개발비가 감소했습니다.

### 2. \*\*GDP 대비 연구개발비 비중\*\*

- \*\*2021 년\*\*: 4.93%
- \*\*2022 년\*\*: 4.93%
- GDP 대비 연구개발비 비중은 변화가 없습니다.

결론적으로, 한국의 연구개발비는 감소했지만, GDP 대비 비중은 불변하여 여전히 높은 수준을

유지하고 있습니다.

다른 국가와 비교하면:

- 미국과 중국은 연구개발비 규모가 증가했습니다.
- 다른 주요 국가들도 대부분 연구개발비 비중에 큰 변화를 보이지 않았습니다.

이러한 데이터는 각국의 연구개발에 대한 투자 방향과 경제 규모에 따라 영향을 받을 수 있습니다.'

---

## 6.2 원인 분석 및 기술적 제언

### 왜 이런 오류가 발생하는가?: 근본 원인 분석과 기술적 제언

#### 근본 원인

##### 1. 공간적 매핑(Spatial Mapping) 능력 부족



모델은 이미지를 픽셀 좌표(x, y)가 아닌 토큰의 시퀀스로 처리하므로, 막대그래프의 높이나 축 라벨을 정확히 매칭하는 데 어려움을 겪음.

##### 2. 작은 텍스트(Small Text) 뭉개짐



이미지 리사이징 과정에서 작은 글씨가 왜곡되어 OCR 성능이 급격히 저하됨.

#### 아키텍트의 제언

- 주의: 높은 정확도가 요구되는 금융/의료 데이터에는 단독 사용을 지양.

- 해결책:  전문 OCR 엔진(예: Tesseract, AWS Textract)과 결합하거나,



- 인간이 검수하는 **Human-in-the-loop** 시스템을 구축.

이러한 오류는 LMM 이 이미지를 처리하는 방식에 기인합니다.

1. **공간적 매핑(Spatial Mapping) 능력 부족**: 모델은 이미지를 토큰의 시퀀스로 변환하여 처리하므로, 픽셀 좌표(x, y)에 대한 정확한 인지 능력이 인간보다 떨어집니다. 따라서 막대그래프의 높이나 촘촘하게 배치된 축(Axis)의 라벨을 정확히 매칭하는 데 어려움을 겪습니다.

2. **작은 텍스트(Small Text)의 뭉개짐**: 이미지 리사이징 과정에서 작은 글씨나 숫자는 흐릿해지거나 왜곡되어, OCR 성능이 급격히 저하됩니다. 특히 detail: low 설정이나 낮은 해상도의 원본 이미지는 이러한 현상을 가속화합니다.

따라서 금융 데이터나 의료 차트와 같이 높은 정확도가 요구되는 분야에서는 GPT-4o 비전 기능을 단독으로 사용하기보다, 전문적인 OCR 엔진(Tesseract, AWS Textract 등)과 결합하거나 사람이 검수하는 Human-in-the-loop 시스템을 구축해야 합니다.

## 7. 자동 퀴즈 생성 에이전트 구축: 프롬프트 엔지니어링 심화

### 7.1 교육용 콘텐츠 생성을 위한 구조화된 프롬프트 설계

#### Step 2 | 분석 결과를 구조화된 콘텐츠로: 프롬프트 엔지니어링

목표: 이미지 분석 결과를 '토익 리스닝 퀴즈'라는 정해진 양식으로 출력하기.

##### 핵심 설계 전략

- 역할 부여 (Role Setup):**  
"토익 리스닝 문제 스타일로..." → 페르소나를 부여하여 톤과 난이도 설정.
- 제약 조건 (Constraint Imposition):**  
"정답은 (1)~(4) 중 하나만 해당하도록", "틀린 것을 고르시오(incorrect)" → 논리적 완결성 확보.
- 형식 지정 (Few-shot Prompting):**  
출력 예시('Q:', '(1)', 'Listening:', '정답:') 제공 → 프로그램이 파싱하기 쉬운 구조화된 결과물 유도.

##### 구조화된 결과물

Q: 이 사진에 대한 설명 중, 틀린 것을 고르시오.  
(1) 한 여성이 그릇에 계란을 풀고 있다.  
(2) 여성은 검은색 운동복을 입고 있다.  
(3) 여성의 표정은 매우 행복해 보인다.  
(4) 여성은 주방이 아닌 거실에서 요리하고 있다.

Listening: Which of the following descriptions of the image is incorrect?

정답: (4)

이미지 분석 능력을 응용하여 영어 듣기 평가 문제를 자동으로 생성하는 프로젝트를 진행합니다. 이 과정의 핵심은 모델이 원하는 형식(Format)으로 정확하게 결과를 출력하도록 유도하는 프롬프트 엔지니어링입니다.

##### Plaintext

제공한 이미지를 바탕으로, 다음과 같은 양식으로 퀴즈를 만들어 주세요.

정답은 (1)~(4) 중 하나만 해당하도록 출제하세요.

토익 리스닝 문제 스타일로 문제를 만들어 주세요.

(중략)

Listening: Which of the following descriptions of the image is incorrect?

- **Role Setup:** "토익 리스닝 문제 스타일"이라는 페르소나를 부여하여 문제의 난이도와 어조를 설정합니다.
- **Constraint Imposition:** "정답은 하나만", "부정형 질문(incorrect)" 등의 제약 조건을 통해 문제의 논리적 완결성을 확보합니다.
- **Few-shot Prompting:** 예시(Example)를 제공하여 출력 형식을 Q:, (1), Listening:, 정답: 등으로 구조화합니다. 이는 추후 프로그램이 텍스트를 파싱(Parsing)할 때 오류를 최소화하는 결정적인

역할을 합니다.

## 7.2 파이썬을 이용한 에러 핸들링과 배치 처리

실제 서비스 환경에서는 API 호출 실패나 모델의 응답 거부 등 다양한 예외 상황이 발생할 수 있습니다. 강의 코드는 try-except 블록과 재귀 호출(Recursive Call)을 이용한 재시도(Retry) 로직을 구현하여 시스템의 견고성(Robustness)을 높였습니다.

```
def image_quiz(image_path, n_trial=0, max_trial=3):  
    if n_trial >= max_trial:  
        raise Exception("Failed to generate a quiz.")  
    try:  
        # API 호출 및 응답 처리  
        ...  
    except Exception as e:  
        print("failed\n" + str(e))  
        return image_quiz(image_path, n_trial+1) # 재귀적 재시도
```

이 코드는 일시적인 네트워크 오류나 모델의 오작동 시 최대 3 회까지 자동으로 재시도함으로써, 대량의 이미지(glob 으로 수집된 파일들)를 일괄 처리하는 배치 작업(Batch Job)이 중간에 중단되지 않도록 보장합니다.

### 01\_image\_quiz.ipynb

```
from glob import glob # 추후 for 문으로 여러 파일의 경로를 가져오기 위해 선언  
from openai import OpenAI  
from dotenv import load_dotenv  
import os  
import base64  
  
load_dotenv()  
api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키 가져오기  
client = OpenAI(api_key=api_key) # OpenAI 클라이언트의 인스턴스 생성
```

```
def encode_image(image_path):
    with open(image_path, "rb") as image_file:
        return base64.b64encode(image_file.read()).decode("utf-8")

def image_quiz(image_path):
    base64_image = encode_image(image_path) # 이미지를 base64 로 인코딩

    quiz_prompt = """
    제공된 이미지를 바탕으로, 다음과 같은 양식으로 퀴즈를 만들어주세요.
    정답은 1~4 중 하나만 해당하도록 출제하세요.
    아래는 예시입니다.
    ----- 예시 -----
```

Q: 다음 이미지에 대한 설명 중 옳지 않은 것은 무엇인가요?

- (1) 베이커리에서 사람들이 빵을 사고 있는 모습이 담겨 있습니다.
- (2) 맨 앞에 서 있는 사람은 빨간색 셔츠를 입고 있습니다.
- (3) 기차를 타기 위해 줄을 서 있는 사람들이 있습니다.
- (4) 점원은 노란색 티셔츠를 입고 있습니다.

정답: (4) 점원은 노란색 티셔츠가 아닌 파란색 티셔츠를 입고 있습니다.

(주의: 정답은 1~4 중 하나만 선택되도록 출제하세요.)

=====

"""

```
messages = [
    {
        "role": "user",
        "content": [
            {"type": "text", "text": quiz_prompt},
```

```

        {
            "type": "image_url",
            "image_url": {
                "url": f"data:image/jpeg;base64,{base64_image}",
            },
        },
    ],
}
]

```

```

response = client.chat.completions.create(
    model="gpt-4o", # 응답 생성에 사용할 모델을 지정
    messages=messages # 대화 기록을 입력으로 전달
)

```

```

return response.choices[0].message.content

```

```

q = image_quiz("../data/images/busan_dive.jpg")
print(q)

```

```

txt = " # ① 문제들을 계속 붙여 나가기 위해 빈 문자열 선언

```

```

no = 1 # 문제 번호를 위해 선언

```

```

for g in glob('../data/images/*.jpg'): # ②

```

```

    try:

```

```

        q = image_quiz(g) # 문제 출제 (가끔 GPT 에서 에러 발생할 수 있으므로 try 문 사용)

```

```

    except Exception as e:

```

```

        print(e)

```

```

        continue

```

```

divider = f'## 문제 {no}\n\n'
print(divider)

txt += divider # ③
# 파일명 추출해 이미지 링크 만들기
filename = os.path.basename(g) # ③ 마크다운에 표시할 이미지 파일 경로 설정
txt += f'![image]({filename})\n\n' # ③

# 문제 추가
print(q)
txt += q + '\n\n-----\n\n'
# ④ 마크다운 파일로 저장
with open('../data/images/image_quiz.md', 'w', encoding='utf-8') as f:
    f.write(txt)

no += 1 # 문제 번호 증가

```

---

## 02\_image\_quiz.ipynb

---

```

from glob import glob # 추후 for 문으로 여러 파일의 경로를 가져오기 위해 선언
from openai import OpenAI
from dotenv import load_dotenv
import os
import base64

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키 가져오기
client = OpenAI(api_key=api_key) # OpenAI 클라이언트의 인스턴스 생성

def encode_image(image_path):
    with open(image_path, "rb") as image_file:

```

```
return base64.b64encode(image_file.read()).decode("utf-8")
```

```
def image_quiz(image_path, n_trial=0, max_trial=3):  
    if n_trial >= max_trial: # 최대 시도 회수에 도달하면 포기  
        raise Exception("Failed to generate a quiz.")
```

```
base64_image = encode_image(image_path) # 이미지를 base64 로 인코딩
```

```
quiz_prompt = """
```

제공된 이미지를 바탕으로, 다음과 같은 양식으로 퀴즈를 만들어주세요.

정답은 1~4 중 하나만 해당하도록 출제하세요.

토익 리스닝 문제 스타일로 문제를 만들어주세요.

아래는 예시입니다.

----- 예시 -----

Q: 다음 이미지에 대한 설명 중 옳지 않은 것은 무엇인가요?

- (1) 베이커리에서 사람들이 빵을 사고 있는 모습이 담겨 있습니다.
- (2) 맨 앞에 서 있는 사람은 빨간색 셔츠를 입고 있습니다.
- (3) 기차를 타기 위해 줄을 서 있는 사람들이 있습니다.
- (4) 점원은 노란색 티셔츠를 입고 있습니다.

Listening: Which of the following descriptions of the image is incorrect?

- (1) It shows people buying bread at a bakery.
- (2) The person standing at the front is wearing a red shirt.
- (3) There are people lining up to take a train.
- (4) The clerk is wearing a yellow T-shirt.

정답: (4) 점원은 노란색 티셔츠가 아닌 파란색 티셔츠를 입고 있습니다.

(주의: 정답은 1~4 중 하나만 선택되도록 출제하세요.)

```
=====
```

```
"""
```

```
messages = [  
    {  
        "role": "user",  
        "content": [  
            {"type": "text", "text": quiz_prompt},  
            {  
                "type": "image_url",  
                "image_url": {  
                    "url": f"data:image/jpeg;base64,{base64_image}",  
                },  
            },  
        ],  
    }  
]
```

```
try:  
    response = client.chat.completions.create(  
        model="gpt-4o", # 응답 생성에 사용할 모델 지정  
        messages=messages # 대화 기록을 입력으로 전달  
    )  
except Exception as e:  
    print("failed\n" + e)  
    return image_quiz(image_path, n_trial+1)
```

```
content = response.choices[0].message.content
```

```
if "Listening:" in content:
```

```

        return content, True
    else:
        return image_quiz(image_path, n_trial+1)

q = image_quiz("../data/images/busan_dive.jpg")
print(q)

txt = " # ① 문제들을 계속 붙여 나가기 위해 빈 문자열 선언
no = 1 # 문제 번호를 위해 선언
for g in glob('../data/images/*.jpg'): #
    q, is_succeed = image_quiz(g)

    if not is_succeed:
        continue

    divider = f'## 문제 {no}\n\n'
    print(divider)

    txt += divider #
    # 파일명 추출해 이미지 링크 만들기
    filename = os.path.basename(g) # ③ 마크다운에 표시할 이미지 파일 경로 설정
    txt += f'![image]({filename})\n\n' # ③

    # 문제 추가
    print(q)
    txt += q + '\n\n-----\n\n'

    # ④ 마크다운 파일로 저장
    with open('../data/images/image_quiz_eng.md', 'w', encoding='utf-8') as f:
        f.write(txt)

```

no += 1 # 문제 번호 증가

---

### 03\_image\_quiz.ipynb

---

```
from glob import glob
import json
from openai import OpenAI
from dotenv import load_dotenv
import os
import base64

load_dotenv()
api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키 가져오기
client = OpenAI(api_key=api_key) # OpenAI 클라이언트의 인스턴스 생성

def encode_image(image_path):
    with open(image_path, "rb") as image_file:
        return base64.b64encode(image_file.read()).decode("utf-8")

def image_quiz(image_path, n_trial=0, max_trial=3):
    if n_trial >= max_trial: # 최대 시도 회수에 도달하면 포기
        raise Exception("Failed to generate a quiz.")

    base64_image = encode_image(image_path) # 이미지를 base64 로 인코딩

    quiz_prompt = """
제공된 이미지를 바탕으로, 다음과 같은 양식으로 퀴즈를 만들어주세요.
정답은 1~4 중 하나만 해당하도록 출제하세요.
토익 리스닝 문제 스타일로 문제를 만들어주세요.
```

아래는 예시입니다.

----- 예시 -----

Q: 다음 이미지에 대한 설명 중 옳지 않은 것은 무엇인가요?

- (1) 베이커리에서 사람들이 빵을 사고 있는 모습이 담겨 있습니다.
- (2) 맨 앞에 서 있는 사람은 빨간색 셔츠를 입고 있습니다.
- (3) 기차를 타기 위해 줄을 서 있는 사람들이 있습니다.
- (4) 점원은 노란색 티셔츠를 입고 있습니다.

Listening: Which of the following descriptions of the image is incorrect?

- (1) It shows people buying bread at a bakery.
- (2) The person standing at the front is wearing a red shirt.
- (3) There are people lining up to take a train.
- (4) The clerk is wearing a yellow T-shirt.

정답: (4) 점원은 노란색 티셔츠가 아닌 파란색 티셔츠를 입고 있습니다.

(주의: 정답은 1~4 중 하나만 선택되도록 출제하세요.)

=====

"""

```
messages = [  
    {  
        "role": "user",  
        "content": [  
            {"type": "text", "text": quiz_prompt},  
            {  
                "type": "image_url",  
                "image_url": {  
                    "url": f"data:image/jpeg;base64,{base64_image}"  
                },  
            },  
        ],  
    },  
]
```

```

        ],
    }
]
try:
    response = client.chat.completions.create(
        model="gpt-4o", # 응답 생성에 사용할 모델 지정
        messages=messages # 대화 기록을 입력으로 전달
    )
except Exception as e:
    print("failed\n" + e)
    return image_quiz(image_path, n_trial+1)

content = response.choices[0].message.content

if "Listening:" in content:
    return content, True
else:
    return image_quiz(image_path, n_trial+1)

# q = image_quiz("./chap06/data/images/busan_dive.jpg")
# print(q)

txt = " # 문제들을 계속 붙여 나가기 위해 빈 문자열 선언
eng_dict = []
no = 1 # 문제 번호를 위해 선언
for g in glob('./data/images/*.jpg'): # ②
    q, is_succeed = image_quiz(g)

    if not is_succeed:
        continue

```

```

divider = f'## 문제 {no}\n\n'
print(divider)

txt += divider
# 파일명 추출해 이미지 링크 만들기
filename = os.path.basename(g) # 마크다운에 표시할 이미지 파일 경로 설정
txt += f'![image]({filename})\n\n'

# 문제 추가
print(q)
txt += q + '\n\n-----\n\n'
# 마크다운 파일로 저장
with open('../data/images/image_quiz_eng.md', 'w', encoding='utf-8') as f:
    f.write(txt)

# 영어 문제만 추출
eng = q.split('Listening: ')[1].split('정답:')[0].strip()

eng_dict.append({
    'no': no,
    'eng': eng,
    'img': filename
})

# json 파일로 저장
with open('../data/images/image_quiz_eng.json', 'w', encoding='utf-8') as f:
    json.dump(eng_dict, f, ensure_ascii=False, indent=4)

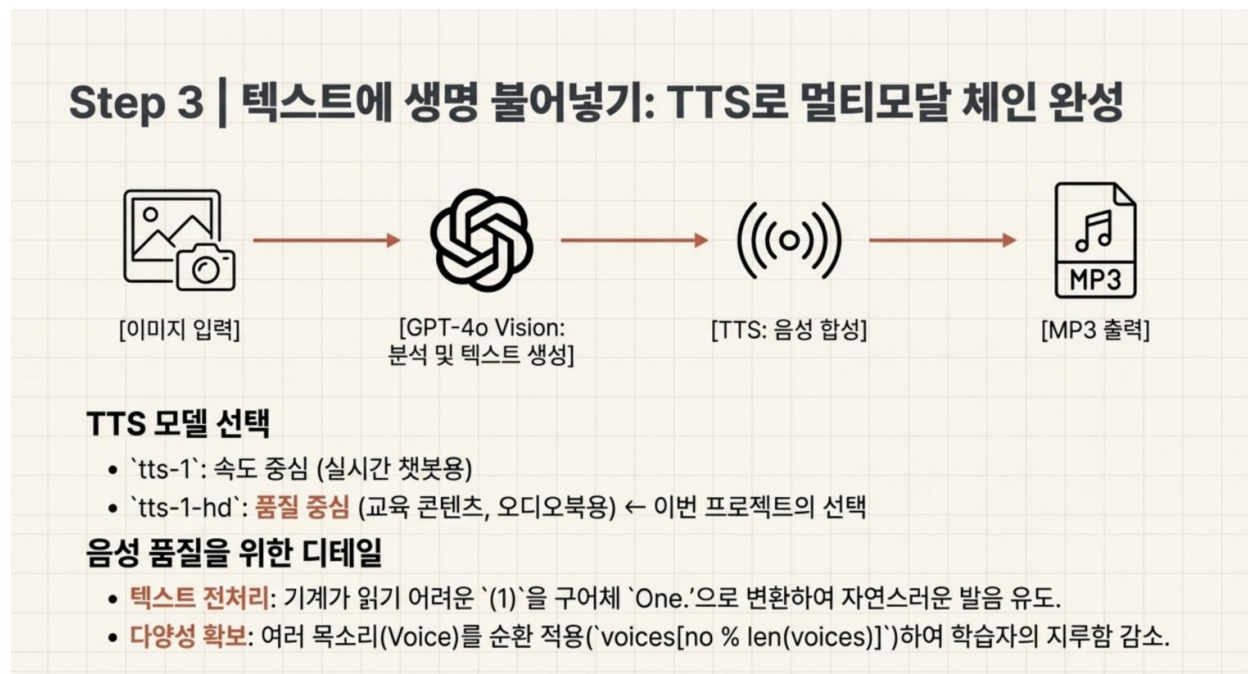
no += 1 # 문제 번호 증가

```

---

## 8. TTS(Text-to-Speech)를 활용한 멀티모달 콘텐츠 완성

### 8.1 OpenAI TTS 모델의 기술적 비교: tts-1 vs tts-1-hd



텍스트로 생성된 퀴즈를 실제 리스닝 평가 자료로 변환하기 위해 OpenAI 의 TTS API 를 활용합니다. 이때 모델 선택은 서비스의 품질을 좌우하는 중요한 요소입니다.

| 특성     | tts-1 (Standard)           | tts-1-hd (High Definition)  |
|--------|----------------------------|-----------------------------|
| 최적화 목표 | 초저지연(Low Latency) 속도 중심    | 고음질(High Quality) 오디오 품질 중심 |
| 샘플링 속도 | 빠름 (실시간 대화형 서비스에 적합)       | 다소 느림 (사전 제작 콘텐츠에 적합)       |
| 음질 특성  | 간혹 잡음(Static)이나 기계적인 느낌 존재 | 선명하고 풍부한 대역폭, 안정적인 발음       |
| 추천 용도  | AI 챗봇, 실시간 통역              | 오디오북, 교육 자료, 방송 내레이션        |

본 강의의 실습 목표는 고품질의 교육 자료(MP3 파일)를 제작하는 것이므로, 처리 속도가 조금 느리더라도 음질이 우수한 **tts-1-hd** 모델을 채택합니다.

### 8.2 음성 합성의 자연스러움을 위한 텍스트 전처리

TTS 엔진은 텍스트를 있는 그대로 읽기 때문에, 사람이 눈으로 읽을 때와 귀로 들을 때의 차이를 고려한 전처리(Preprocessing)가 필수적입니다. 예를 들어, 문제 번호 (1)을 TTS 가 "Parenthesis one"이나 무음으로 처리할 수 있습니다. 이를 방지하기 위해 강의 코드는 replace 함수를 사용하여

명시적인 구어체 텍스트로 변환합니다.

---

# 텍스트 치환을 통한 발음 교정

```
quiz = quiz.replace("- (1)", "- One.Wt")
```

```
quiz = quiz.replace("- (2)", "- Two.Wt")
```

---

또한 Wt (탭 문자)나 마침표를 적절히 삽입하여 문장 간의 휴지기(Pause)를 줌으로써, 학습자가 문제를 듣고 생각할 시간을 확보할 수 있도록 유도합니다.

### 8.3 다양한 화자(Voice) 적용을 통한 다양성 확보

OpenAI TTS 는 alloy, ash, coral, echo, fable, onyx, nova, sage, shimmer 등 다양한 음색(Voice Persona)을 제공합니다.<sup>17</sup> 실습 코드에서는 문제 번호(no)에 따라 화자를 순환시키는 로직(voices[no % len(voices)])을 적용하여, 매 문제마다 다른 성우가 읽어주는 듯한 효과를 구현했습니다. 이는 학습자의 지루함을 덜고 다양한 억양과 톤에 익숙해지게 하는 교육적 효과를 제공합니다.

## 04\_tts.ipynb

---

```
from openai import OpenAI
```

```
from dotenv import load_dotenv
```

```
import os
```

```
load_dotenv()
```

```
api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키를 가져옵니다.
```

```
client = OpenAI(api_key=api_key) # OpenAI 클라이언트의 인스턴스를 생성합니다.
```

---

```
response = client.audio.speech.create(
```

```
    model="tts-1-hd",
```

```
    voice="alloy",
```

```
    input="Hello world! This is a TTS test.",
```

```
)
```

```
response.write_to_file("hello_world.mp3")
```

```
# 재생
```

```
import IPython.display as ipd
```

```
ipd.Audio("hello_world.mp3")
```

---

```
# 다른 목소리
```

```
voice = "ash"
```

```
mp3_file = f"hello_world_{voice}.mp3"
```

```
response = client.audio.speech.create(  
    model="tts-1-hd",  
    voice=voice,  
    input=f"Hello world! I'm {voice}. This is a TTS test.",  
)
```

```
response.write_to_file(mp3_file)
```

```
# 재생
```

```
import IPython.display as ipd
```

```
ipd.Audio(mp3_file)
```

---

```
import json
```

```
# json 파일 열기
```

```
with open('./data/images/image_quiz_eng.json', 'r', encoding='utf-8') as f:  
    eng_dict = json.load(f)
```

```
eng_dict
```

---

```
voices = ['alloy', 'ash', 'coral', 'echo', 'fable', 'onyx', 'nova', 'sage', 'shimmer']
```

---

```
for q in eng_dict:
```

```
    no = q['no']
```

```
    quiz = q['eng']
```

```
    quiz = quiz.replace("- (1)", "- One.Wt")
```

```
    quiz = quiz.replace("- (2)", "- Two.Wt")
```

```
    quiz = quiz.replace("- (3)", "- Three.Wt")
```

```
    quiz = quiz.replace("- (4)", "- Four.Wt")
```

```
    print(no, quiz)
```

```
voice = voices[no % len(voices)] # 문제 개수를 목소리 개수로 나눈 나머지 값으로 선택
```

```
response = client.audio.speech.create(
```

```
    model="tts-1-hd",
```

```
    voice=voice,
```

```
    input=f'#{no}. {quiz}',
```

```
)
```

```
response.write_to_file(f"../data/audio/{no}.mp3")
```

---

```
ipd.Audio(f"../data/audio/1.mp3")
```

---

## 9. 결론 및 향후 발전 방향

### 9.1 강의 요약 및 성과

본 강의를 통해 우리는 GPT-4o 라는 강력한 멀티모달 모델을 활용하여, 시각 정보를 해석하고(Vision), 이를 교육적 맥락에 맞게 재가공하며(LLM), 최종적으로 음성 정보로 변환하는(TTS) 완전한 자동화 파이프라인을 구축했습니다.

- **기술적 성취:** Base64 인코딩부터 REST API 통신, 프롬프트 엔지니어링, TTS 모델 튜닝까지 AI 애플리케이션 개발의 핵심 기술을 실습했습니다.
- **실무적 통찰:** 비전 모델의 환각 현상과 해상도 이슈 등 현존하는 기술적 한계를 명확히 인지하고, 이를 보완하기 위한 전략(Human-in-the-loop, 고해상도 모드 사용 등)을 학습했습니다.

### 9.2 차세대 멀티모달 에이전트의 전망

현재의 시스템은 정지 이미지(Static Image) 처리에 머물러 있지만, 향후 gpt-4o-realtime 과 같은 모델이 보편화되면 실시간 비디오 스트림을 분석하여 즉각적으로 음성 피드백을 주는 '동시통역 에이전트'나 '실시간 시각 보조 도구'로 발전할 것입니다. 본 강의에서 다룬 데이터 처리 원칙과 프롬프트 설계 전략은 이러한 미래 기술을 받아들이는 견고한 토대가 될 것입니다.